

From Point-clouds to Heightfields: Enhancing Humanoid Robot Perception with Multi-Camera Sensing

Ana Jaramillo, Summer Undergraduate Research Fellow (SURF) proposal in the laboratory of Prof. Aaron Ames

Abstract

The AMBER Lab at Caltech conducts research on locomotion, control, and human-robot interaction, aiming to build robots that can safely operate in human environments. Recently, the lab has been working with the Unitree Humanoid G1, a humanoid designed for research in robotics and human-robot interaction. Ensuring safety in humanoid operation requires robust perception, as the robot must reliably sense and respond to its environment. This project focuses on enhancing perception of the G1 robot and bridging the gap between simulation and real-world deployment through the development of a data processing pipeline that translates multi-camera point-cloud data into one physical heightfield. Results show smooth integration of three RGB-D cameras as well as high speed processing time and real-time visualization. Future work will focus on new head designs implementing new pipeline to further improve perception, robustness, and safety.

Video link: <https://youtu.be/D5BvHe2A-70>

Introduction

Humanoid robot design has been of great interest in the robotic community [1]. This area seeks to create robots that resemble human function to later have the potential to operate in human-centered environments and assist with tasks ranging from industrial automation to personal assistance. A critical aspect of successful robot function and safety in unstructured environments is perception, particularly visual perception [2]. Visual sensors provide the robot with information about its surroundings, helping with obstacle avoidance, localization, and human-robot interaction.

Common visual sensing technologies in mobile and humanoid robotics include stereo cameras, RGB-D (Red, Green, Blue + Depth) cameras, LiDAR (Light Detection and Ranging), laser scanners, and proximity sensors [3]. Each sensor type offers tradeoffs in resolution, field of view, depth accuracy, power consumption, and more. Selecting the appropriate visual sensors and a useful design requires knowledge of the robot mobility and desired function.

At Caltech's AMBER Lab, research is currently being conducted using the Unitree G1 humanoid robot. The G1, showed in Figure 1, is a mobile bipedal robot designed to walk and move freely in real-world environments [4]. Considering that the robot is intended to operate in shared spaces with humans, safety is a big concern. To ensure safety during usage and navigation, a robust perception system is needed to detect obstacles, uneven terrain, and humanoid presence in real-time. Currently, the Unitree G1 robot is equipped with an Intel RealSense D435 RGB-D camera angled toward the ground and a 3D LiDAR sensor. The downward camera is useful for terrain adaptation and foot placement, while the LiDAR (Figure 2) aids with 3D mapping.



Figure 1. Unitree G1 robot currently used by the Amber Lab.



Figure 2. Close-up picture of Unitree G1 head using an Intel RealSense D435 camera and 3D LiDAR

While this configuration provides some special mapping, it provides limited coverage and leaves blind spots, constraining the robot's ability to operate safely in shared spaces.

Another challenge arises from the gap between simulation and real-world deployment. Simulated environments are idealized—for example, staircases in simulation are perfectly geometric—whereas physical sensors capture noisy or incomplete data. This mismatch limits the transferability of results from simulation to hardware. Thus, expanding the G1's perception system serves a dual purpose: improving safety in real-world operation and strengthening the bridge between simulation and physical sensing.

This project addresses these challenges by explore a multi-camera sensing approach coupled with a real-time point-cloud-to-heightfield processing pipeline. By converting point-cloud data to heightfields, we get a clear representation of how the data is getting interpreted by the robot. This allows both enhanced situational awareness for the robot and more realistic comparisons between simulated and physical experiments.

Results

Sensor Mount Mechanical Desing

The sensor mount successfully supported up to four cameras. The final assembly allowed for stable placement of three RGB-D cameras, enabling consistent geometric transformations and alignment for the resulting point-clouds

Static Heightfield Validation

With the cameras securely mounted and calibrated, the next step was to test whether their combined point clouds could be fused into a single terrain representation. To begin, a static heightfield was generated in MuJoCo from the three-camera data. This proof-of-concept confirmed that the point-cloud frame transformations were correct, providing confidence that live heightfield generation would be feasible.

Live Heightfield Generation

The final ROS2, C++, and MuJoCo implementation achieved continuous, real-time point-cloud to heightfield conversion, with an average update rate of 51 Hz. When coupled with visualization, the effective end-to-end update frequency remained above 30 Hz, which is sufficient for closed-loop humanoid locomotion. The following video (<https://youtu.be/D5BvHe2A-70>) illustrates the live heightfield dynamically responding to changes in the environment, demonstrating the feasibility of this approach for real-world usage.

Deployment Performance: Jetson vs GPU workstation

To test deployment feasibility, the pipeline was tested on both an embedded NVIDIA Jetson Orin NX Super and a high-performance GPU workstation (Ubuntu 24.04.3 LTS desktop with an AMD Ryzen 9 5950X CPU, 64 GB RAM, and an NVIDIA RTX 4090 GPU). On the Jetson, testing was limited to a single-camera configuration, and the processing pipeline achieved an average update time of 27.0ms (37 Hz), indicating that the core algorithm was capable of near-real-time operation on embedded hardware. However, visualization performance was noticeably lower, subjectively appearing closer to 10–15 Hz. Validating this pipeline on the Jetson is important because the final system ideally runs on the robot's onboard computer.

In contrast, the GPU workstation sustained significantly faster processing (mean 11.3ms, 89 Hz) and maintained smooth visualization exceeding 30 Hz, even with multi-camera configurations. These results highlight the trade-off between portability and computational capability: the Jetson offers an embedded deployment path but with reduced effective visualization rates, whereas the workstation easily handled real-time operation with multiple cameras (Figure 3).

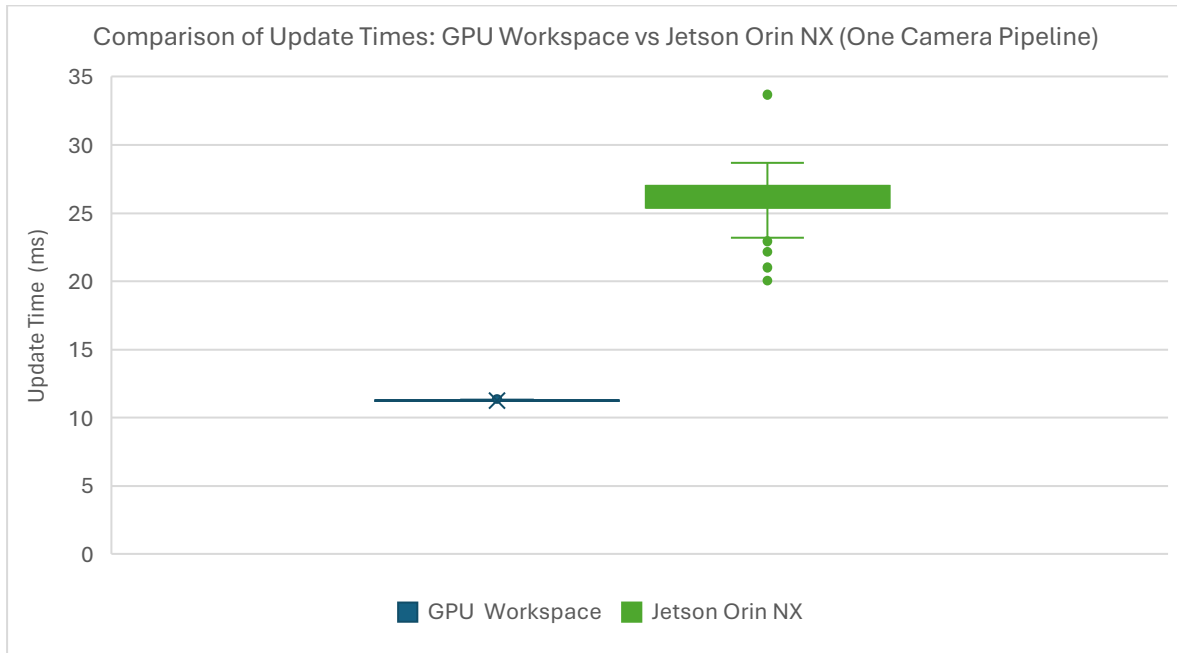


Figure 3. Data distribution of data processing update times for the GPU workplace pipeline (Blue) versus the Jetson Orin NX Super (Green).

Methods

Sensor Mount Mechanical Desing

To support the multi-camera sensing, a custom camera holder was designed to physically mount the sensors in a configuration that optimized field of view. The first prototype was a four-way camera holder, designed in SolidWorks and 3D-printed (Figure 4) in PLA. This was a crucial step because a rigid, well-defined mount was required to accurately determine the geometric transformations for each camera, ensuring that the point clouds aligned correctly in the robot's coordinate frames.

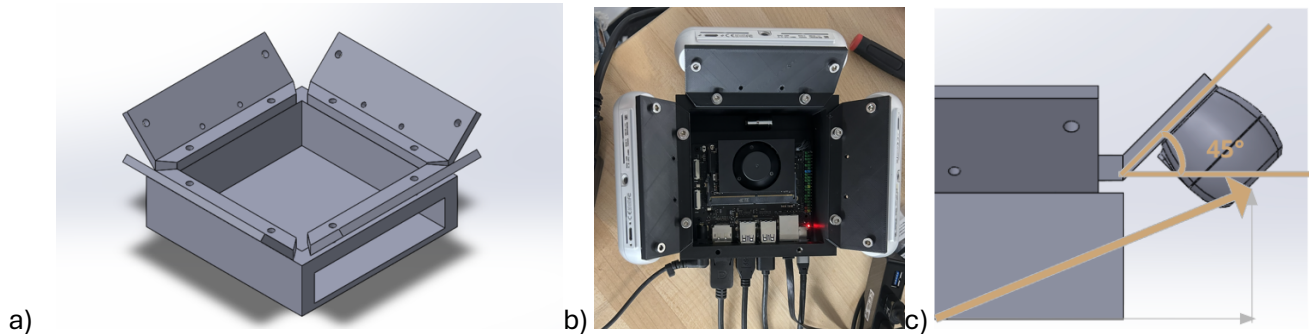


Figure 4. a) SolidWorks CAD design for 4-camera holder. b) 3D printed mount fully assembled and equipped with all three cameras and computer. c) Schematic for transformations needed for successful data orientation.

Once the holder was fabricated and the cameras were mounted, the position and orientation of each camera was measured relative to an arbitrary origin located at the center of the holder's base. Distances in the x, y, and z directions, as well as roll, pitch, and yaw, were recorded. These values were then incorporated into the launch file as static transformations, allowing the point clouds to be displayed in RViz2 with realistic alignment to the cameras' true orientations.

Point-Cloud Validation via Static Heightfield

After achieving multi-camera point cloud visualization in RViz2, the next objective was to represent the data as a MuJoCo heightfield. The full data processing was done on the Jetson Orin NX Super using transmission control protocol (TCP) to transfer the data to the computer where the MuJoCo heightfield visualization ran. The script on the Jetson subscribed to the topic, cropped the data to fit a chosen grid size, and mapped each grid cell to a single height value by taking the maximum z-coordinates of all points within the cell. The resulting values were normalized to produce the grayscale PNG image required by MuJoCo. Using this PNG as input, a heightfield was successfully visualized through the *geom hfield* feature (Figure 5). This process helped validate the pipeline necessary to create the live heightfield.

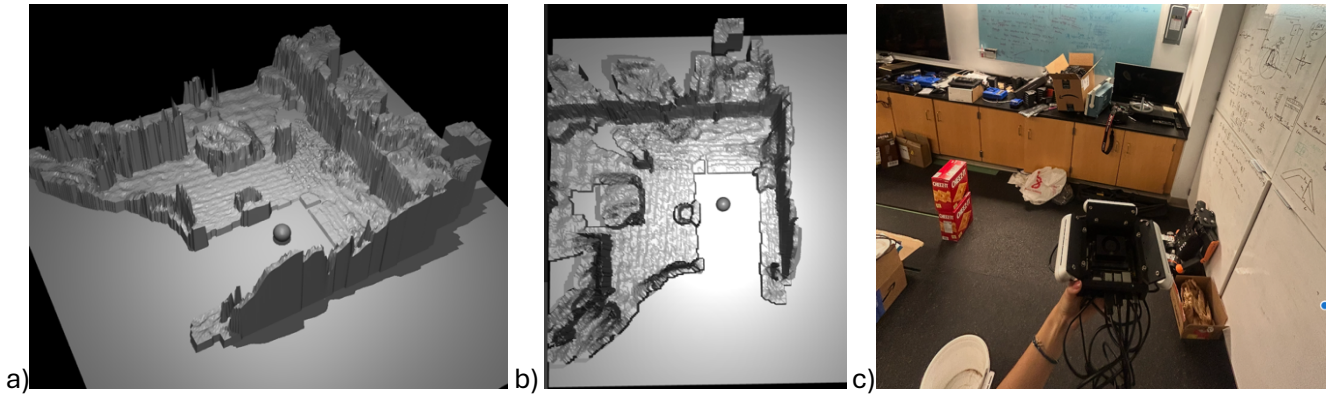


Figure 5. a,b) Same MuJoCo heightfield visualization, different viewer angles. c) Real life point of view for the heightfield produced

Software Framework

The perception system was implemented in ROS 2, with launch files coordinating the initialization of multiple camera drivers, transform broadcasters, and the heightfield processing node. Each Intel RealSense (D455) RGB-D camera was launched as a separate node, publishing a PointCloud2 topic on a unique namespace (e.g., /topic_1/cam_1/depth/color/points). To fuse data from multiple sensors, spatial relationships were defined using tf2. Each camera frame was linked to an arbitrary base frame (the center of the sensor mount), representing the robot's base frame, through static transforms, allowing the heightfield node to subscribe to multiple point-clouds in a common coordinate system.

Each camera was launched as a separate node, publishing its own PointCloud2 topic and its own static transform. These topics were fused by a dedicated processing node that generated the heightfield representation. Figure 6 illustrates this architecture.

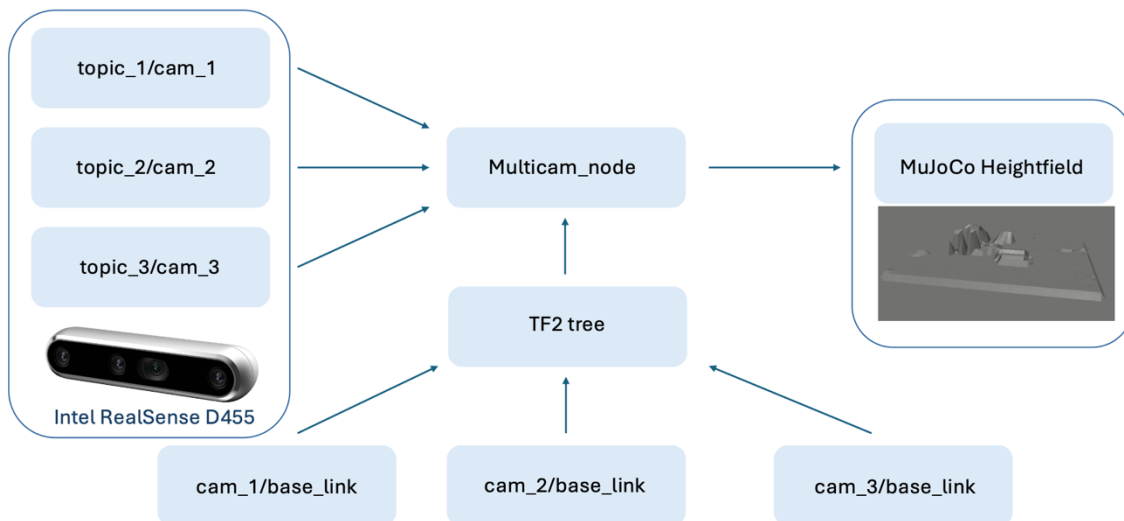


Figure 6. High level diagram of software framework showing ROS2 nodes and generic data flow.

Point-cloud to Heightfield Algorithm

We designed a custom algorithm that converts incoming PointCloud2 messages from multiple cameras into a singular heightfield in MuJoCo. Each camera is associated with a per-frame buffer, a previous-frame buffer, and a timestamp flag to track message freshness. The final output is stored in an array and the normalized field is passed to MuJoCo.

On reception of a point-cloud message, the algorithm queries the TF buffer to obtain the transform into 'base_link', computes the rotation and translation, and applies them to all valid points. The transformed 3D points are then fused into the camera's grid representation using a helper function that assigns the maximum height to each grid cell.

To suppress frame-to-frame noise, we apply low-pass filtering with coefficient $\alpha=0.3$. For each grid cell, the current frame value is blended with the previous frame and missing values are filled from the prior frame. This ensures smooth continuity.

At each timer tick, the algorithm checks that all cameras are within a freshness window (WINDOW_S). If so, their per-camera grids are fused together using maximum height values across cameras. The combined grid is then normalized from metric heights into the range $[0,1]$ using the global parameters Z_{min} and Z_{max} .

Finally, the finalized array is uploaded into the MuJoCo model via `mjr_uploadHField`, producing a continuously updated terrain that reflects the fused perception of all cameras.

Figure 7 shows the end-to-end pipeline, while the complete pseudocode overview is provided in Appendix A.

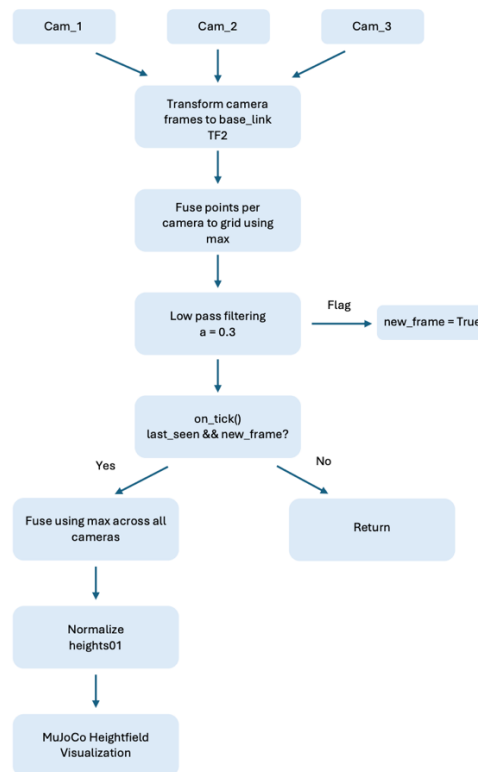


Figure 7. Pipeline overview for multi-cam point-cloud fusion and heightfield generation

Deployment Pipelines

We used two environments to execute this pipeline. Both run on ROS2 and use C++, with Intel RealSense D455 cameras and TF2 providing transforms to a base link.

Jetson to Workstation. On the Jetson Orin NX (no MuJoCo installed), the node computes the final array exactly as described in the Algorithm subsection. The resulting array is sent over TCP to a workstation, which updates the MuJoCo heightfield and renders it. This keeps perception on-device while outsourcing visualization.

Workstation-only. On the GPU workstation, the identical node produces the final array and uploads it directly to MuJoCo in the same process. This configuration was used for development and validation but is also equivalent to the Jetson pipeline.

Across both deployments, topics, static transforms, grid/ROI parameters, freshness window (WINDOW_S), and normalization bounds (Z_MIN, Z_MAX) are identical; only the transport of the array differs.

References

- [1] S. Saeedvand, M. Jafari, H. S. Aghdasi, y J. Baltes, “A comprehensive survey on humanoid robot development”, *Knowl. Eng. Rev.*, vol. 34, p. e20, ene. 2019, doi: 10.1017/S0269888919000158.
- [2] L. Wijayathunga, A. Rassau, y D. Chai, “Challenges and Solutions for Autonomous Ground Robot Scene Understanding and Navigation in Unstructured Outdoor Environments: A Review”, *Appl. Sci.*, vol. 13, núm. 17, Art. núm. 17, ene. 2023, doi: 10.3390/app13179877.
- [3] A. Bonci, P. D. Cen Cheng, M. Indri, G. Nabissi, y F. Sibona, “Human-Robot Perception in Industrial Environments: A Survey”, *Sensors*, vol. 21, núm. 5, Art. núm. 5, ene. 2021, doi: 10.3390/s21051571.
- [4] “Humanoid robot G1_Humanoid Robot Functions_Humanoid Robot Price | Unitree Robotics”. Consultado: el 31 de julio de 2025. [En línea]. Disponible en: <https://www.unitree.com/g1>
- [5] “Using the Intel® RealSense™™ Depth cameras D4xx in Multi-Camera Configurations”, Intel. Consultado: el 31 de julio de 2025. [En línea]. Disponible en: <https://www.intel.com/content/www/us/en/content-details/841998/using-the-intel-realsense-depth-cameras-d4xx-in-multi-camera-configurations.html>

Acknowledgements

I’d like to thank my mentors Adrian Boedtke Ghansah and Sergio Esteban for their time and patience, as well as Professor Aaron Ames.

I’d like to thank Mikaela Laite for the welcoming introduction to the AMBER Lab and all her attention. This work was possible thanks to the financial support of the Carol Casey as a Carol Casey Surf Fellow and the Caltech SURF Program.

Appendix A


```

// This algorithm converts multiple camera pointcloud streams into a
normalized heightfield for live rendering in MuJoCo.
Class MultiCamNode { // "multicam_node"
    // DATA
    grid_cam[i] // meters, current frame from camera i
    grid_cam_prev[i] // meters, previous frame from camera i
    last_seen[i] // timestamp of last message from camera i
    heights01 // normalized [0-1] field to render

    // Start ROS2 node
    construct TF buffer+listener
    for each CAM_TOPICS[i], subscribe with callback()
    create periodic timer every 25ms (30 Hz)

    //Callback
    void callback(){
        tf = lookupTransform()
        last_seen_[cam_idx] // record freshness of message
        // Build rotation and translation matrix
        R = quat_to_rot(tf.rotation)
        (tx, ty, tz) = tf.translation
        // Extract XYZ points from Pointcloud2
        pts_xyz <-- []
        for each point p in msg {
            if p.x,y,z are finite {
                append pts_xyz += [p.x, p.y, p.z]
            }
        }
        Transform all points to base_link

        // Fuse points to grid keeping the maximum point per grid
        fuse_points_to_grid_meters(pts_xyz, grid_cam_[cam_idx]
        // assigns the maximum z value for each grid cell

        // Low-pass filtering,  $\alpha = 0.3$ 
        curr = grid_cam[cam_idx]
        prev = grid_cam_prev[cam_idx]
        for k in curr.size() {
            if curr[k] && prev[k] {
                curr[k] =  $\alpha * curr[k] + (1.0 - \alpha) * prev[k]$ 
            }
            else if (!curr[k] && prev[k]) {

```



```

        curr[k] = prev[k]
    }
    prev[k] = curr[k]
}
new_frame = TRUE // indicates fresh frame
}
void on_tick() {
    // verify all cameras are fresh enough. Fresh window: WINDOW_S
    // ensures obstacles are preserved across all cameras.
    for each camera {
        if last_seen[i] == now - last_seen[i] > WINDOW_S {
            return
        }
        if !new_frame {
            return
        }
    }

    Fuse per-camera grids by max to a fused cell or custom fill to
minimum (Z_MIN = -1.0)
    // Normalize meters
    v = (v - Z_MIN) / (Z_MAX - Z_MIN)
    heights01 // data sent to MuJoCo

}
}
main() {
    // Continuously updates the MuJoCo heightfield with the fused, filtered,
normalized grid
    xml = make_model_xml()
    while window not closed {
        set_heightfield
        mjr_uploadHField(m, con, hid)
        mjr_render()
    }
}
}

```